Debugging with LLMs: Good and Bad Points

Large Language Models (LLMs) help fix code. They find mistakes fast by seeing patterns. They guess where the problem is and suggest small fixes that keep the main idea. They are good at finding common mistakes, like off-by-one errors, state reset problems, mixed-up variables, and number overflows. If you give an LLM a small function, it can "think" through steps (like "1-(2+3)" for a calculator). This shows the problem, and then it can find a one-line fix.

In this task, we found and fixed five one-line mistakes:
Basic Calculator: The sign did not reset inside parentheses. So, the inside expression got the outside sign. This made it negative twice at the end. Fix: Reset sign to +1 after saving the state.
Special Character Counting II: A backward check used an index $gt 0$. This missed index 0. It did not find a capital letter at the start. Fix: Change to index $>= 0$.
Minimum Cost Good Title: The rule for breaking ties used old letters to set new letters. This caused a sorting mistake. Fix: Set `newLetter` to `currentNewLetter`.
Robot Collision: In the collision loop, a left-moving robot was pushed. This stopped it from hitting a right-moving robot in front too early. Fix: Only push the robot if no right-moving robot is on top.
Maximum Prime Substring Sum: The prime loop used `divisor * divisor`. This could overflow with a big number. Fix: Change to `long` inside the loop limits.

Good Points We Saw:
Speed: LLM quickly finds bad lines. It suggests good one-line code fixes.
Finding Errors: It finds common kinds of mistakes without needing all tests.
Explanation:** The fixes come with simple reasons. This helps people check them.

Bad Points and Problems We Saw:
Not Clear and Overfitting: If the rules are not clear, LLM may "play it safe." It might change what the code does. For example, it first limited substring length (a speed change). This changed the result. Then it found the right overflow fix (changing type inside the loop).
Cannot Run Code: When tests cannot run, checking the fix means thinking hard. This can miss special cases.
Tricky Rules: Some logic (like in comparators or state machines) can look right but be wrong overall. LLM may need hints or examples to find the real problem.